

Introducción a las Redes Neuronales Convolucionales

Conceptos Fundamentales y Arquitectura

Dr. Rubén Hernández Torres

18 de mayo de 2026

Contenido

- 1 Introducción
- 2 Componentes Clave
- 3 Arquitectura Clásica
- 4 Ventajas y Aplicaciones
- 5 ¿Qué son los Modelos Preentrenados?
- 6 Modelos Más Usados en Visión Artificial
- 7 ¿Qué son los Modelos Out-of-the-Box?
- 8 Detección de Objetos en Tiempo Real
- 9 Segmentación Avanzada
- 10 Modelos Optimizados para Edge
- 11 Visión-Lenguaje y Modelos Fundacionales
- 12 OpenMMLab: Ecosistema Completo
- 13 OpenCV DNN Zoo
- 14 Recomendaciones de Aplicación
- 15 Referencias

¿Qué son las Redes Neuronales Convolucionales?

Definition (Red Neuronal Convolutacional (CNN))

Una **Red Neuronal Convolutacional** es una clase de redes neuronales profundas, comúnmente aplicada al análisis de datos con topología tipo grilla, como imágenes. Su arquitectura se basa en la operación matemática de *convolución* en lugar de la multiplicación de matrices general [1].

Inspiración Biológica

Las CNNs están inspiradas en el proceso visual de la corteza cerebral de los animales, específicamente en las **células simples y complejas** descubiertas por Hubel y Wiesel (1962).

Capas Fundamentales de una CNN

- 1 **Capa Convolutacional:** Aplica filtros (kernels) a la entrada para extraer características.
- 2 **Capa de Activación:** Introduce no linealidad (habitualmente ReLU).
- 3 **Capa de Pooling (Muestreo):** Reduce dimensionalidad espacial (MaxPooling / AveragePooling).
- 4 **Capa Fully Connected (Completamente Conectada):** Realiza la clasificación final.

Importancia

La combinación de estas capas permite a la red aprender jerarquías de características: desde bordes y texturas hasta objetos completos.

Operación de Convolución

Definition (Convolución Discreta 2D)

Para una imagen de entrada I y un kernel K , la convolución S se define como:

$$S(i, j) = (I * K)(i, j) = \sum_m \sum_n I(i + m, j + n) K(m, n)$$

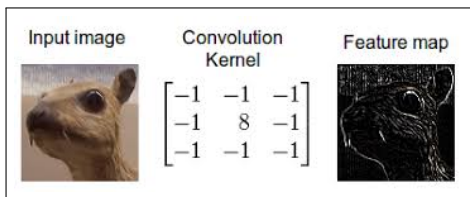


Figura: Esquema de convolución: kernel deslizándose sobre la imagen.

Parámetros Clave en Convolución

- **Kernel / Filtro:** Matriz pequeña de pesos (ej. 3×3 , 5×5).
- **Stride (Paso):** Número de píxeles que se desplaza el kernel.
- **Padding (Relleno):** Adición de ceros en los bordes para controlar el tamaño de salida.
- **Profundidad:** Número de filtros aplicados (canales de salida).

Arquitectura LeNet-5 (LeCun et al., 1998)

Una de las primeras CNNs para reconocimiento de dígitos escritos a mano (MNIST).

- C1: Conv (6 filtros 5x5)
- S2: Pooling (2x2)
- C3: Conv (16 filtros 5x5)
- S4: Pooling
- C5: Conv 120
- F6: Fully connected (84)
- Output: Softmax (10)

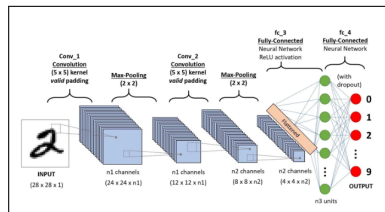


Figura: Arquitectura LeNet-5 [2].

Evolución de Arquitecturas Modernas

Año	Arquitectura	Innovación Clave
1998	LeNet-5	Primera CNN práctica
2012	AlexNet	ReLU, Dropout, GPU
2014	VGG	Bloques de convoluciones 3x3
2014	GoogleNet (Inception)	Módulos Inception
2015	ResNet	Conexiones residuales

Cuadro: Hitos en el desarrollo de CNNs.

Ventajas frente a Redes Fully Connected

Problema de FC para imágenes:

- Una imagen 256×256 RGB $\rightarrow$ 196,608 entradas.
- Primera capa oculta con 1000 neuronas $\rightarrow$ **196 millones de parámetros.**
- Sobreajuste masivo.

Ventajas de CNN:

- **Compartición de pesos:** Un mismo filtro se aplica en toda la imagen.
- **Invarianza traslacional.**
- Reducción drástica de parámetros.
- Captura de jerarquías espaciales.

Visión por Computadora

- Clasificación de imágenes (ImageNet)
- Detección de objetos (YOLO, Faster R-CNN)
- Segmentación semántica (U-Net, Mask R-CNN)
- Reconocimiento facial

Más allá de imágenes

- Procesamiento de video
- Análisis de señales (ECG, audio)
- Procesamiento de lenguaje natural (texto como secuencia 1D)

Definición y Analogía

Definition (Modelo Preentrenado)

Un **modelo preentrenado** es una red neuronal que ha sido entrenada en un conjunto de datos a gran escala (como ImageNet, COCO o LAION) para aprender representaciones visuales generales.

Analogía

Es como contratar a un **graduado en arte** que ya sabe identificar bordes, texturas, colores y formas básicas. Solo necesita aprender tu tarea específica, no empezar desde cero.

¿Por qué funcionan?

Los modelos preentrenados capturan **jerarquías de características**:

- Capas tempranas: bordes, colores, texturas
- Capas medias: formas, patrones, partes de objetos
- Capas profundas: conceptos semánticos, objetos completos

Ventajas de Usar Modelos Preentrenados

Métrica	Desde cero	Preentrenado
Tiempo de entrenamiento	Semanas	Horas o días
Datos necesarios	100k-1M+ imágenes	100-5000 imágenes
Rendimiento típico	Variable	Excelente (SOTA)
Recursos computacionales	Múltiples GPUs	1 GPU o CPU

Impacto práctico:

- Reduce costos de infraestructura significativamente
- Democratiza el deep learning
- Acelera iteración y experimentación

Modelos Populares (2026) - Detección de Objetos

Modelo	mAP COCO	Latencia	Característica
YOLOv12-X	55.2 %	1.64-11.8ms	Atención eficiente, sin NMS
RF-DETR-M	54.7 %	4.52ms	Transformer, >60 mAP en RF100-VL
RTMDet-XL	52.8 %	~3.3ms	300+ FPS

Cuadro: Modelos de detección en tiempo real

Segmentación

- **SAM 2 (Meta)**: Segmentación promptable en imágenes y video
- **Grounded SAM**: Detección open-vocabulary + SAM

Modelo	Parámetros	Top-1 ImageNet	Uso
C-RADIOv4-H	653M	83.09 %	Embeddings (zero-shot)
ViT-L/14	303M	~85 %	Tareas generales
ResNet50	25M	76-80 %	Balance recurso/-rendimiento
DINOv2	1.1B	-	Características auto-supervisadas

- **timm** (Hugging Face): >1000 modelos (ResNet, ViT, EfficientNet, ConvNeXt, Swin)
- **Modelos multi-modales**: Florence-2, Qwen3-VL, Molmo

Modelos Listos para Usar

Definition (Modelo Out-of-the-Box)

Modelos preentrenados que pueden ser utilizados directamente en aplicaciones de visión artificial sin necesidad de entrenamiento adicional, solo con mínimos pasos de preprocesamiento.

Ventajas Clave

- **Implementación rápida:** Integración en minutos, no semanas
- **Infraestructura mínima:** Muchos funcionan en CPU o GPUs modestas
- **Sin datos de entrenamiento:** Zero-shot capabilities
- **Open Source:** Sin costos de licencia, código disponible

Consideraciones

- Licencias (Apache 2.0, MIT, AGPL-3.0) [?]
- Posible fine-tuning para dominios específicos

YOLOv12 - El Estado del Arte en Detección

Características:

- Arquitectura attention-centric
- Latencia ultrabaja (1.64 ms en T4 GPU)
- Precisión SOTA: 40.6 % mAP (modelo Nano)
- Versiones: detección, segmentación, clasificación

Modelos disponibles:

Modelo	mAP	Latencia (ms)
YOLOv12-N	40.6 %	1.64
YOLOv12-S	48.0 %	2.61
YOLOv12-M	52.5 %	4.86
YOLOv12-X	55.2 %	11.79

YOLOv12 - Segmentación y Clasificación

Instance Segmentation

Modelo	mAPbox	mAPmask
YOLOv12n-seg	39.9 %	32.8 %
YOLOv12s-seg	47.5 %	38.6 %
YOLOv12m-seg	52.4 %	42.3 %

Clasificación de Imágenes

Modelo	Top-1 Acc	Latencia (ms)
YOLOv12n-cls	71.7 %	1.27
YOLOv12s-cls	76.4 %	1.52
YOLOv12x-cls	80.1 %	3.64

Fuente: YOLOv12 GitHub Repository [?]

OpenMMLab MMDetection

- Detector totalmente convolutional
- 52.8 % mAP en COCO
- 300+ FPS en NVIDIA 3090
- Soporte para rotated object detection

Ventajas:

- Mejor trade-off parámetros/precisión
- Tamaños: tiny a extra-large
- Integración con MMDetection

SAM 2 - Segment Anything (Meta)

Características Principales

- Segmentación promptable en imágenes y video
- Memoria streaming para tracking en video
- Zero-shot generalization
- Soporte para múltiples objetos simultáneamente

Modelos SAM 2.1:

- sam2.1_hiera_tiny
- sam2.1_hiera_small
- sam2.1_hiera_base_plus
- sam2.1_hiera_large

¿Qué es OpenCV DNN Module?

- Motor de inferencia de alto rendimiento
- Soporta TensorFlow, PyTorch, ONNX
- Optimizado para CPUs y hardware embebido
- Soporte FP16, OpenVINO, CUDA

Modelos disponibles en OpenCV Zoo:

Tarea	Modelos
Detección facial	YuNet, NanoDet, YOLOX
Segmentación humana	PP-HumanSeg, EfficientSAM
Reconocimiento facial	SFace, FaceRecognizer
Detección placas	LPD_YuNet
OCR	PPOCR-Det (Chino/Inglés)

Modelos Ligeros para Dispositivos Edge

Opciones para despliegue en CPU/Edge

- **YuNet**: Detección facial rápida y precisa
- **NanoDet**: Detector de objetos ultraligero
- **PP-HumanSeg**: Segmentación de personas optimizada
- **EfficientSAM**: Segmentación eficiente basada en SAM

Rendimiento en ARM (Jetson Nano, Raspberry Pi)

- Optimización con TIM-VX, OpenVINO, CANN
- Modelos cuantizados INT8 para NPUs
- Benchmarks disponibles en OpenCV Zoo

Características:

- Text-Image Pretraining with Spatial Awareness
- Excelente en tareas espaciales (segmentación, depth)
- Variantes: B/14 (85.7M), L/14 (303M), G/14 (1.1B)
- Licencia: Apache 2.0

Métricas destacadas:

- PASCAL segmentation: hasta 85.2 %
- NYU-depth: 0.339 (error)
- Flickr image→text: 95.4 %

Modelos Open Source para Tareas Específicas

Modelo	Tarea Principal	Licencia
Florence-2 (Microsoft)	Captioning, grounding, OCR	MIT
Qwen3-VL (Alibaba)	OCR, razonamiento visual	Apache 2.0
Molmo (Ai2)	Grounding visual	Apache 2.0
Detectron2 (Meta)	Detección + segmentación	Apache 2.0
Grounding DINO + SAM	Segmentación open-vocabulary	Apache 2.0

Cuadro: Modelos Open Source listos para producción

Características del Ecosistema

[citation:7]

- **Diseño modular:** Combina componentes fácilmente
- **Múltiples tareas:** Detección, segmentación, panóptica, semisupervisada
- **Alta eficiencia:** Operaciones BBOX/Mask en GPU
- **Modelos SOTA:** RTMDet, DINO, Conditional-DETR, DiffusionDet

Modelos Disponibles

Modelo	mAP COCO	FPS (TRT FP16)
RTMDet-tiny	41.8 %	515
RTMDet-x	52.8 %	322
RTMDet-x-instance	44.6 % (mask)	188

Modelos destacados:

- **YuNet**: Detección facial (WIDER Face)
- **SFace**: Reconocimiento facial
- **PP-HumanSeg**: Segmentación personas
- **LPD_YuNet**: Detección placas vehículos
- **WeChatQRCode**: QR codes
- **PPOCR-Det**: Detección texto (Chino/Inglés)
- **VitTrack**: Tracking de objetos
- **EfficientSAM**: Segmentación eficiente

Guía de Selección por Escenario

Escenario	Recomendación	Justificación
Tiempo real (video, robótica)	YOLOv12, RTMDet	Latencia <10ms, 30+ FPS
Máxima precisión (análisis médico)	Fine-tune de ViT o C-RADIO	Embeddings ricos + datos específicos
Sin GPU (edge, web)	MobileNetV4, EfficientNet-Lite	Optimizados para inferencia ligera
Datos muy limitados (<100)	Zero-shot (YOLO-World, SAM2)	Aprovecha conocimiento general
Segmentación detallada	SAM2 + fine-tuning	Máscaras precisas

Referencias Bibliográficas

-  Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press. Capítulo 9: Arquitecturas convolucionales.
-  LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 2278-2324.
-  Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet classification with deep convolutional neural networks. *Advances in Neural Information Processing Systems*, 25, 1097-1105.
-  He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition* (pp. 770-778).
-  Chollet, F. (2017). *Deep Learning with Python*. Manning Publications.